lec11

April 24, 2026

1 Algebraic Types

Beyond inductively defined lists, we can define other data types. An example is the binary tree type.

1.1 Binary trees

A binary tree is a data type that can be defined inductively as:

[49]:
```
type 'a tree = Empty | Node of 'a * 'a tree * 'a tree
```

[49]:
```
type 'a tree = Empty | Node of 'a * 'a tree * 'a tree
```

An example of a binary tree with the following shape is:

```
   2
  / \
 1   3
```

[50]:
```
let t0 = Node(2,
              Node(1,
                   Empty,
                   Empty),
              Node(3,
                   Empty,
                   Empty))
```

[50]:
```
val t0 : int tree = Node (2, Node (1, Empty, Empty), Node (3, Empty, Empty))
```

To insert an element into a binary tree in OCaml we generate the binary tree corresponding to the insertion of the element. We do it inductively on the structure of the tree. There are many alternatives for inserting an element into a binary tree. One of them is the following:

[ ]:
```
let rec insert x t =
  match t with
  | Empty -> Node(x, Empty, Empty)
  | Node(y,l,r) -> Node(x,t,Empty)
```

```
let _ = assert (insert 0 t0 = Node (0, Node (2, Node (1, Empty, Empty), Node␣
 ↪(3, Empty, Empty)), Empty))
```

1.2 Binary search trees

A binary search tree is a binary tree with the following property: for each node, the value of the node is greater than all values in the left subtree and less than all values in the right subtree. Additionally, the left subtree and the right subtree are also binary search trees.

Inserting an element into a binary search tree while maintaining the binary search tree property is done as follows:

[57]:
```
let rec insert_bst x t =
  match t with
  | Empty -> Node(x, Empty, Empty)
  | Node(y,l,r) -> if x < y then Node(y,insert_bst x l,r) else Node(y,l,␣
 ↪insert_bst x r)

let t1 = insert_bst 0 t0
let t2 = Node(4, t0, Empty)
let _ = assert(t1 = Node (2, Node (1, Node (0, Empty, Empty), Empty), Node (3,␣
 ↪Empty, Empty)))
```

[57]:
```
val insert_bst : 'a -> 'a tree -> 'a tree = <fun>
```

[57]:
```
val t1 : int tree =
  Node (2, Node (1, Node (0, Empty, Empty), Empty), Node (3, Empty, Empty))
```

[57]:
```
val t2 : int tree =
  Node (4, Node (2, Node (1, Empty, Empty), Node (3, Empty, Empty)), Empty)
```

[57]:
```
- : unit = ()
```

Let us define an auxiliary function `remove_min` that removes the node with the smallest value from a binary search tree. This function will be useful for removing a node from the binary search tree. The `remove_min` function returns the binary search tree without the node with the smallest value.

[ ]:
```
(** [remove_min t] denotes the tree removing the minimum value from [t]
    pre: [t <> Empty] *)
let rec remove_min t =
  match t with
  | Empty -> failwith "Precondition violated!"
  | Node(x,Empty,r) -> r
  | Node(x,l,r) -> Node(x, remove_min l, r)
```

```
(** [min_tree t] denotes the smaller value in [t]
    pre: [t <> Empty] *)
let rec min_tree t =
  match t with
  | Empty -> failwith "Precondition violated!"
  | Node(x,Empty,r) -> x
  | Node(x,l,r) -> min_tree l

(** [min_tree t] denotes the larger value in [t]
    pre: [t <> Empty] *)
let rec max_tree t =
  match t with
  | Empty -> failwith "Precondition violated!"
  | Node(x,_,Empty) -> x
  | Node(_,_,r) -> max_tree r
```

Given these auxiliary functions, we can define a function that corresponds to the inductive proof of the binary search tree property. If we consider that the recursive call to the left subtree and to the right subtree corresponds to the induction hypothesis (the property for a smaller tree), then we can define the result for the composed tree by checking whether all elements in the left subtree are smaller than the root and all elements in the right subtree are greater than the root.

[ ]:
```
let rec is_a_bst t =
  match t with
  (* Base case *)
  | Empty -> true

  (* Base case *)
  | Node(_, Empty,Empty) -> true

  (* Inductive case if the left subtree is empty *)
  | Node(x,Empty,r) -> (* by h.i. *) is_a_bst r && min_tree r >= x

  (* Inductive case if the right subtree is empty *)
  | Node(x,l,Empty) -> (* by h.i. *) is_a_bst l && max_tree l < x

  (* General Inductive case *)
  | Node(x,l,r) -> (* by h.i. *) is_a_bst l && (* by h.i. *) is_a_bst r &&␣
 ↪max_tree l < x && min_tree r >= x

let _ = assert(is_a_bst t0)
let _ = assert(is_a_bst t1)
let _ = assert(is_a_bst t2)
```

Removing a value from a binary search tree is done by searching for the value and removing it. When an intermediate node is removed, it must be replaced by the minimum value of the right subtree (or the maximum value of the left subtree). This is done by calling the `remove_min` function on the right subtree and replacing the intermediate node with the result. The left subtree remains

unchanged. The result is a new binary search tree with the same properties as before, but without the removed value.

[73]:
```
let rec remove x t =
  match t with
  | Empty -> Empty
  | Node(y,l,r) ->
    if x = y then
      Node(min_tree r, l, remove_min r)
    else
      if x < y then
        Node(y, remove x l ,r)
      else
        Node(y, l, remove x r)

let _ = assert (remove 2 t0 = Node (3, Node (1, Empty, Empty), Empty))
let _ = assert (remove 2 t2 = Node (4, Node (3, Node (1, Empty, Empty), Empty),␣
 ↪Empty))
```

```
File "[73]", line 6, characters 20-21:
6 |       Node(min_tree r, l, remove_min r)
                        ^

Error: This expression has type 'a tree/1
       but an expression was expected of type 'b tree/2
       File "[49]", line 1, characters 0-53:
         Definition of type tree/1
       File "[32]", line 1, characters 0-53:
         Definition of type tree/2
```

- binary trees
- insert in a bst
- remove in a bst (remove_min)
- tree_size
- depth
- pre_order
- in_order
- post_order
- map
- fold_in_order
- fold_pre_order
- fold_post_order

An alternative to the `remove_min` function above is to compute the minimum value at the same time as it is removed. The `remove` function is then defined as follows:

Other functions on trees that we can define are the following:

[53]:
```
let rec tree_size t =
  match t with
  | Empty -> 0
  | Node(_, l, r) -> 1 + (tree_size l) + (tree_size r)

let rec search_bst x t =
  match t with
  | Empty -> false
  | Node(y, l, r) -> x = y || (x < y && (search_bst x l)) || search_bst x r

let rec tree_depth t =
  match t with
  | Empty -> 0
  | Node(_, l, r) -> 1 + max (tree_depth l) (tree_depth r)

let _ = assert (4 = tree_size t2)
let _ = assert (3 = tree_depth t2)
```

[53]:
```
val tree_size : 'a tree -> int = <fun>
```

[53]:
```
val search_bst : 'a -> 'a tree -> bool = <fun>
```

[53]:
```
val tree_depth : 'a tree -> int = <fun>
```

[53]:
```
- : unit = ()
```

[53]:
```
- : unit = ()
```

[ ]:
```
let rec tree_size t =
  match t with
  | Empty -> 0
  | Node(_, l, r) -> 1 + tree_size l + tree_size r


let rec tree_depth t =
  match t with
  | Empty -> 0
  | Node(_,l,r) -> 1 + max (tree_depth l) (tree_depth r)

let _ = assert (4 = tree_size t2)
let _ = assert (3 = tree_depth t2)
```

2 Binary Tree Traversals

Tree traversals are functions that traverse the tree in a specific way. There are three types of binary tree traversals:

- prefix (pre-order): visits the root, then the left subtree, then the right subtree
- infix (in-order): visits the left subtree, then the root, then the right subtree
- postfix (post-order): visits the left subtree, then the right subtree, then the root

[ ]:
```
let t = Node(1,Empty,Node(2,Empty,Node (3,Empty,Empty)))
let t3 = Node(0, t, t)

let rec pre_order t =
  match t with
  | Empty -> []
  | Node(x,l,r) ->
      x :: (pre_order l) @ (pre_order r)

let _ = pre_order t3
let _ = pre_order t2
```

[ ]:
```
val t : int tree = Node (1, Empty, Node (2, Empty, Node (3, Empty, Empty)))
```

[ ]:
```
val t3 : int tree =
  Node (0, Node (1, Empty, Node (2, Empty, Node (3, Empty, Empty))),
   Node (1, Empty, Node (2, Empty, Node (3, Empty, Empty))))
```

[ ]:
```
val pre_order : 'a tree -> 'a list = <fun>
```

[ ]:
```
- : int list = [0; 1; 2; 3; 1; 2; 3]
```

[ ]:
```
- : int list = [4; 2; 1; 3]
```

[59]:
```
let rec in_order t =
  match t with
  | Empty -> []
  | Node(x,l,r) ->
      (in_order l) @ [x] @ (in_order r)

let _ = in_order t2
```

[59]:
```
val in_order : 'a tree -> 'a list = <fun>
```

[59]:
```
- : int list = [1; 2; 3; 4]
```

[62]:
```
let rec post_order t =
  match t with
  | Empty -> []
  | Node(x,l,r) ->
      (post_order l) @ (post_order r) @ [x]

let _ = post_order t2
```

[62]:
```
val post_order : 'a tree -> 'a list = <fun>
```

[62]:
```
- : int list = [1; 3; 2; 4]
```

[ ]:
```
let rec pre_order t =
  match t with
  | Empty -> []
  | Node(x,l,r) -> x::(pre_order l)@(pre_order r)

let _ = assert([0;1;2;3;1;2;3] = pre_order t3)
```

[ ]:
```
let rec in_order t =
  match t with
  | Empty -> []
  | Node(x,l,r) -> (in_order l)@[x]@(in_order r)

let _ = assert( [1;2;3;0;1;2;3] = in_order t3)
```

[63]:
```
let ot =␣
 ↪Node(4,Node(2,Node(1,Empty,Empty),Node(3,Empty,Empty)),Node(6,Node(5,Empty,Empty),Node(7,Em

let _ = assert( [1; 2; 3; 4; 5; 6; 7] = in_order ot)
let _ = assert( [4; 2; 1; 3; 6; 5; 7] = pre_order ot)
```

[63]:
```
val ot : int tree =
  Node (4, Node (2, Node (1, Empty, Empty), Node (3, Empty, Empty)),
   Node (6, Node (5, Empty, Empty), Node (7, Empty, Empty)))
```

[63]:
```
- : unit = ()
```

[63]:
```
- : unit = ()
```

[ ]:
```
let rec post_order t =
  match t with
  | Empty -> []
  | Node(x,l,r) -> (post_order l)@(post_order r)@[x]

let _ = assert( [1; 3; 2; 5; 7; 6; 4] = post_order ot)
```

[ ]:
```
let rec in_order_rev t =
  match t with
  | Empty -> []
  | Node(x,l,r) -> (in_order_rev r)@[x]@(in_order_rev l)

let _ = assert( [7; 6; 5; 4; 3; 2; 1] = in_order_rev ot)
```

3 Higher-order Functions (Map and Fold)

[65]:
```
let rec map_lst f l =
  match l with
  | [] -> []
  | x::xs -> f x :: map_lst f xs

let map_opt f o =
  match o with
  | None -> None
  | Some x -> Some (f x)

let _ = assert (map_opt (fun x -> x + 1) (Some 1) = Some 2)
let _ = assert (map_opt (fun x -> x + 1) None = None)

let _ = assert (map_lst ((+) 1) [1;2;3] = [2;3;4])
```

[65]:
```
val map_lst : ('a -> 'b) -> 'a list -> 'b list = <fun>
```

[65]:
```
val map_opt : ('a -> 'b) -> 'a option -> 'b option = <fun>
```

[65]:
```
- : unit = ()
```

[65]:
```
- : unit = ()
```

[65]:
```
- : unit = ()
```

[67]:
```
let rec map f t =
  match t with
  | Empty -> Empty
  | Node(x,l,r) -> Node(f x, map f l, map f r)

let _ = ot
let _ = map (fun x -> x+1) ot
```

[67]:
```
val map : ('a -> 'b) -> 'a tree -> 'b tree = <fun>
```

[67]:
```
- : int tree =
Node (4, Node (2, Node (1, Empty, Empty), Node (3, Empty, Empty)),
 Node (6, Node (5, Empty, Empty), Node (7, Empty, Empty)))
```

[67]:
```
- : int tree =
Node (5, Node (3, Node (2, Empty, Empty), Node (4, Empty, Empty)),
 Node (7, Node (6, Empty, Empty), Node (8, Empty, Empty)))
```

[ ]:
```
let rec map f t =
  match t with
  | Empty -> Empty
  | Node(x,l,r) -> Node(f x, map f l, map f r)

let _ = map string_of_int ot
let _ = map ((+)1) ot
```

[ ]:
```
let map_opt f o =
  match o with
  | None -> None
  | Some v -> Some (f v)

let _ = map_opt string_of_int (Some 1)
let _ = map_opt string_of_int None
let _ = map_opt ((+)1) (Some 1)
let _ = map_opt ((+)1) None
```

[72]:
```
let rec fold_in_order f acc t =
  match t with
  | Empty -> acc
  | Node(x, l, r) ->
      let acc2 = fold_in_order f acc l in
      let acc3 = f acc2 x in
      fold_in_order f acc3 r

let _ = List.fold_left (fun acc x -> acc+x) 0 [1;2;3]
```

```
let _ = fold_in_order (+) 0 ot

let _ = fold_in_order (fun acc x -> acc@[x]) [] ot
let _ = List.rev (fold_in_order (fun acc x -> x::acc) [] ot)
```

[72]: val fold_in_order : ('a -> 'b -> 'a) -> 'a -> 'b tree -> 'a = <fun>

[72]: - : int = 6

[72]: - : int = 28

[72]: - : int list = [1; 2; 3; 4; 5; 6; 7]

[72]: - : int list = [1; 2; 3; 4; 5; 6; 7]

[ ]:
```
let rec fold_left f acc l =
  match l with
  | [] -> acc
  | x::xs -> fold_left f (f acc x) xs

let _ = fold_left (+) 0 (pre_order ot)

let rec fold_in_order f acc t =
  match t with
  | Empty -> acc
  | Node(x,l,r) -> let acc_l = fold_in_order f acc l in let acc_root = f acc_l␣
 ↪x in fold_in_order f acc_root r

let _ = fold_in_order (+) 0 ot
let _ = fold_in_order (fun acc x -> acc@[x]) [] ot
let filter p t = fold_in_order (fun acc x -> if p x then acc@[x] else acc) [] ot

let _ = filter (fun x -> x mod 2 = 0) ot
```

[ ]:
```
let rec fold_pre_order f acc t =
  match t with
  | Empty -> acc
  | Node(x,l,r) -> let acc_root = f acc x in let acc_l = fold_pre_order f␣
 ↪acc_root l in fold_pre_order f acc_l r

let _ = fold_pre_order (fun acc x -> acc@[x]) [] ot
```